ПРАКТИЧЕСКИЙ СПРАВОЧНИК / 2026

Git через Bash и консоль

Понятные команды, ежедневные сценарии и безопасное исправление ошибок — от первого репозитория до совместной разработки.

НАСТРОЙКА	ЕЖЕДНЕВНАЯ РАБОТА	КОМАНДНАЯ РАБОТА
имя, почта, репозиторий	status → add → commit	branch → pull → push

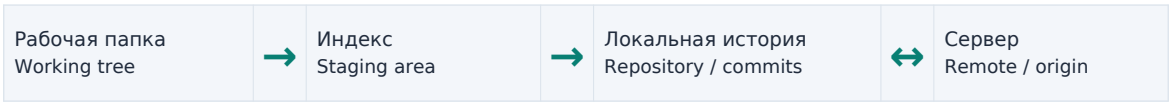
Для начинающих и тех, кому нужна надёжная шпаргалка.

```
git status
git add .
git commit -m
"update"
```

01 / ОСНОВЫ

Как устроен Git

Git хранит историю изменений. Он не сохраняет каждый файл как отдельную копию проекта, а связывает снимки состояния (коммиты) в историю.



Зона	Что это значит	Команды
Рабочая папка	Файлы, которые вы сейчас редактируете. Изменения ещё не подготовлены к коммиту.	git status git diff
Индекс (staging)	Список изменений, которые войдут в следующий коммит.	git add git diff --staged
Локальная история	Уже сохранённые снимки проекта и их авторы/сообщения.	git commit git log
Удалённый репозиторий	Общая копия на сервере: GitHub, GitLab или другой хост.	git fetch git pull git push

Коротко о Bash

Команда	Назначение	Пример
`pwd`	Показать текущую папку	pwd
ls -la	Показать файлы, включая скрытые	ls -la
`cd путь`	Перейти в папку	cd ~/projects/app
`cd ..`	На уровень выше	cd ..
mkdir имя	Создать папку	mkdir demo

Главный ритуал

Сначала посмотрите, что происходит: `git status`. Эта команда почти всегда безопасна и помогает выбрать следующий шаг.

02 / СТАРТ

Установка и первый репозиторий

Команды настройки обычно выполняются один раз на компьютере, а команды создания или клонирования — для каждого проекта.

1. Проверьте, установлен ли Git

```
git --version
git help -a
```

2. Настройте имя и email автора коммитов

```
git config --global user.name "Ваше Имя"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global --list
```

Используйте email, который вы хотите видеть в истории проекта. Параметр `--global` сохраняет настройку для текущего пользователя. Для одного проекта можно задать значения без `--global`, находясь внутри его папки.

3. Начать новый проект или скачать существующий

Сценарий	Последовательность
Новая папка и новый репозиторий	1. `mkdir demo` 2. `cd demo` 3. `git init` 4. Создайте файлы проекта
Скачать существующий проект	`git clone URL` Замените URL адресом репозитория. Git создаст папку и настроит удалённый адрес `origin`.

Проверить состояние проекта	`cd имя-папки`
	`git status`
	`git remote -v`

Полезные настройки терминала Git

```
git config --global core.editor "code --wait"
git config --global color.ui auto
git config --global --get user.name
```

Важно

Не запускайте `git init` внутри уже существующего Git-проекта без причины. Для обычной работы с GitHub/GitLab чаще всего достаточно `git clone`.

03 / КАЖДЫЙ ДЕНЬ

Цикл: проверить → подготовить → сохранить

Самая важная последовательность в Git. Она помогает не включить в коммит лишние файлы и писать понятную историю.

```
git status
git diff
git add путь/к/файлу
git diff --staged
git commit -m "Кратко: что изменено"
git status
```

Команда	Для чего нужна	Когда использовать
`git status`	Показывает ветку и изменённые, подготовленные и неотслеживаемые файлы.	До и после значимых действий.
`git diff`	Показывает изменения, ещё не добавленные в индекс.	Перед добавлением файлов.
`git add файл`	Добавляет файл в индекс для следующего коммита.	Когда выбрали нужные изменения.
`git add -p`	Позволяет выбирать изменения частями (hunks).	Когда в одном файле несколько разных задач.
`git diff --staged`	Показывает именно то, что попадёт в коммит.	Непосредственно перед commit.
`git commit -m "..."`	Создаёт коммит из содержимого индекса.	Когда изменения образуют логически цельный шаг.

Как писать хорошие сообщения коммитов

Хорошо

```
git commit -m "Добавить проверку email"
git commit -m "Исправить расчёт скидки"
```

Не очень

```
git commit -m "fix"
git commit -m "разное"
```

Запомните

`git add` сам по себе ничего не отправляет на сервер. `git commit` сохраняет изменения локально. Для публикации понадобится `git push`.

Быстрый просмотр истории

```
git log --oneline --decorate -10
git log --oneline --graph --decorate --all
git show --stat HEAD
```

04 / ВЕТКИ

Работа через отдельные ветки

Ветка — отдельная линия истории. Для новой функции или исправления ошибки удобно создать короткоживущую ветку, не смешивая работу с основной.

Обычная последовательность для новой задачи

```
git switch main
git pull --ff-only
git switch -c feature/login-form
# редактируйте файлы и создавайте коммиты
git status
git add src/login.*
git commit -m "Добавить форму входа"
git push -u origin feature/login-form
```

`main` — пример имени основной ветки; в конкретном проекте она может называться иначе. Перед обновлением основной ветки убедитесь, что ваши локальные изменения сохранены или отложены.

Команды для веток

Команда	Описание
`git branch`	Список локальных веток; звёздочка отмечает текущую.
`git branch -a`	Локальные и известные Git удалённые ветки.
`git switch имя`	Переключиться на существующую ветку.
`git switch -c имя`	Создать ветку и перейти в неё.
`git branch -d имя`	Удалить локальную уже слитую ветку.
`git merge имя`	Влить указанную ветку в текущую.
`git log --oneline --graph --all`	Посмотреть разветвление истории.

Как завершить задачу

```
git switch main
git pull --ff-only
git merge --no-ff feature/login-form
git push origin main
git branch -d feature/login-form
```

Командная практика

В команде обычно отправляют ветку на сервер, открывают Pull/Merge Request, проходят проверку и сливают изменения через принятую в проекте процедуру. Не пушьте напрямую в защищённую `main`, если команда так не договорилась.

Merge и rebase — в двух словах

merge объединяет истории и при необходимости создаёт merge-коммит. rebase переносит ваши коммиты поверх другой базы и делает историю линейнее. Не переписывайте общую историю опубликованной ветки без согласования; для новичка безопаснее следовать workflow проекта.

05 / ОБМЕН

Удалённый репозиторий: fetch, pull, push

Удалённый репозиторий — точка обмена с сервером. Обычно `origin` — имя сервера, с которого клонирован проект.

Команда	Что делает	Важно
`git remote -v`	Показывает адреса удалённых репозиториев.	Проверьте URL перед push.
`git fetch origin`	Скачивает объекты и обновляет сведения об удалённых ветках.	Не сливает автоматически изменения с вашей веткой.
`git pull`	Получает изменения и интегрирует их в текущую ветку.	Может выполнить merge или rebase — зависит от настроек.
`git pull --ff-only`	Обновляет ветку только fast-forward-ом.	Завершится ошибкой, если историю надо объединять иначе.
`git push`	Отправляет локальные коммиты в удалённый репозиторий.	Нужны доступ и настроенный upstream.
`git push -u origin имя`	Отправляет ветку и задаёт upstream для следующих push/pull.	Удобно при первой публикации ветки.
`git remote add origin URL`	Добавляет удалённый адрес.	Нужно для локального репозитория без remote.

Сценарий: вы начинаете день в существующем проекте

```
git status
git switch main
git pull --ff-only
git switch -c feature/my-task
```

Сценарий: отправить готовую ветку

```
git status
git log --oneline -5
git push -u origin feature/my-task
```

Сценарий: разобраться, что изменилось на сервере

```
git fetch origin
git log --oneline --graph --decorate HEAD..origin/main
git diff HEAD..origin/main
```

Не путайте

`fetch` обновляет сведения и скачивает данные, но не меняет автоматически текущую ветку. `pull` делает fetch и затем интегрирует полученное. Если вы не уверены в состоянии ветки, начните с `git status` и `git fetch`.

06 / ИСПРАВЛЕНИЕ

Как отменять изменения безопасно

Перед любой отменой остановитесь и выполните `git status`. Разные команды работают на разных этапах: рабочая папка, индекс, коммиты или удалённая история.

Что нужно сделать	Команда	Осторожно
Убрать файл из индекса, но оставить изменения в файле	git restore --staged файл	Не удаляет правки из рабочей папки.
Отбросить незастейдженные изменения файла	git restore файл	Потеряются правки, не добавленные в индекс. Сначала проверьте git diff.
Исправить последний коммит, не опубликованный на сервере	git commit --amend	Пересоздаёт последний коммит. Меняйте опубликованную историю только осознанно.
Создать обратный коммит, отменяющий предыдущий	git revert HASH	Обычно безопаснее для общей опубликованной истории.
Переместить HEAD/ветку назад	git reset ...	Режимы различаются; можно потерять доступ к коммитам/изменениям.
Удалить неотслеживаемые файлы	git clean -n	Это только предварительный просмотр. Без -n команда удаляет файлы.

Три варианта reset

Команда	Эффект в общем случае
`git reset --soft HEAD~1`	Убирает последний коммит, оставляет изменения подготовленными в индексе.
`git reset --mixed HEAD~1`	Убирает последний коммит; изменения остаются в файлах, но снимаются с индекса. Это поведение reset по умолчанию.
`git reset --hard HEAD~1`	Сбрасывает и коммит, и состояние отслеживаемых файлов. Риск потери незакоммиченной работы.

Красная зона

Не запускайте `git reset --hard`, `git clean -fd` или force-push как «попробовать исправить». Сначала выясните, где находятся нужные изменения: `git status`, `git diff`, `git diff --staged`, `git log --oneline --all`.

Если нужно просто отменить изменения, уже опубликованные в общей ветке, предпочтите `git revert`. Так Git создаст новый коммит с обратными изменениями, не переписывая предыдущую историю.

07 / ПОЛЕЗНЫЕ ИНСТРУМЕНТЫ

Stash, конфликты, .gitignore и теги

Временно отложить незавершённые изменения

```
git stash push -m "работа над формой"
git stash list
git stash show -p stash@{0}
git stash apply stash@{0}
git stash drop stash@{0}
```

`stash` полезен, когда нужно переключиться на другую задачу, не создавая коммит. `apply` применяет изменения, сохраняя запись в stash; `pop` применяет и удаляет запись, если применение прошло успешно.

Разрешение конфликта при merge/pull

```
git status
# откройте конфликтные файлы и выберите итоговый вариант
git diff
git add путь/к/исправленному-файлу
git commit
```

Внутри конфликтного файла можно увидеть маркеры `<<<<<<<`, `=======` и `>>>>>>>`. Удалите маркеры, оставьте правильный результат, сохраните файл и убедитесь, что `git diff` показывает ожидаемые изменения. Для отмены незавершённого merge обычно доступна команда `git merge --abort`.

Не добавлять лишнее в репозиторий: .gitignore

```
# пример .gitignore
.env
.venv/
node_modules/
__pycache__/
*.log
dist/
```

Файл `.gitignore` влияет на неотслеживаемые файлы. Он не перестаёт отслеживать файл, который уже был добавлен в Git. Секреты (токены, пароли, приватные ключи) нельзя коммитить: если секрет попал в историю, его нужно отозвать/сменить.

Теги — отметить релиз или важную точку

```
git tag
git tag -a v1.0.0 -m "Релиз 1.0.0"
git push origin v1.0.0
git show v1.0.0
```

Полезная привычка

Перед коммитом посмотрите и на индекс (`git diff --staged`), и на статус (`git status`). Так вы заметите случайно добавленные `.env`, логи, сборки и другие ненужные файлы.

08 / ШПАРГАЛКА

Готовые рабочие последовательности

Можно сохранить эту страницу отдельно: здесь собраны четыре частых сценария, которые закрывают большую часть повседневной работы.

A. Я только что склонировал проект

```
git clone URL
cd имя-проекта
git status
git switch -c feature/my-task
```

B. Я изменил код и хочу сохранить работу локально

```
git status
git diff
git add нужный-файл
git diff --staged
git commit -m "Понятное описание"
```

C. Я хочу отправить задачу на сервер

```
git status
git log --oneline -5
git push -u origin feature/my-task
```

D. Перед началом новой задачи хочу синхронизироваться

```
git status
git switch main
git pull --ff-only
git switch -c feature/next-task
```

Команды, которые стоит знать наизусть

Команда	Вспоминайте, когда…
`git status`	нужно понять текущее состояние
`git diff / --staged`	нужно проверить конкретные изменения
`git add -p`	хотите включить только часть правок
`git log --oneline --graph --all`	нужно разобраться в истории веток
`git switch -c имя`	начинаете отдельную задачу
`git fetch origin`	нужно узнать состояние сервера без слияния
`git revert HASH`	нужно отменить опубликованный коммит безопаснее

Минимум перед push

Проверьте `git status`, просмотрите последние коммиты и убедитесь, что пушите именно свою ветку. Не отправляйте секреты, `.env` и временные артефакты.

09 / НАПОСЛЕДОК

Термины и официальные источники

Термин	Простое объяснение
Repository (репозиторий)	Папка проекта вместе с историей Git.
Commit (коммит)	Зафиксированный снимок подготовленных изменений.
Branch (ветка)	Указатель на линию истории коммитов.
HEAD	Указатель на текущую позицию — обычно текущую ветку.
Remote / origin	Настроенный адрес удалённого репозитория; `origin` — стандартное имя при clone.
Upstream	Удалённая ветка, связанная с локальной; используется по умолчанию при pull/push.
Merge conflict	Ситуация, когда Git не может автоматически выбрать итог для части изменений.
Staging area / index	Подготовленная область: что попадёт в следующий коммит.

Официальная документация

Источник	Зачем открыть	Ссылка
Git Reference	Полный справочник команд	https://git-scm.com/docs
Pro Git — на русском	Книга с подробными объяснениями	https://git-scm.com/book/ru/v2
git-status	Документация по статусу репозитория	https://git-scm.com/docs/git-status
Working with Remotes	Как работать с удалёнными репозиториями	https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes
Branching Workflows	Рекомендуемые подходы к веткам	https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows

Главное правило

Git легче освоить, если не спешить с командами, меняющими историю. Проверьте статус → посмотрите diff → сделайте небольшой коммит → отправьте свою ветку. Если команда кажется опасной, сперва откройте `git help имя-команды` или официальную документацию.

Справочник подготовлен для повседневной работы в Bash, Linux, macOS, Git Bash и терминале IDE. Некоторые команды и названия веток зависят от настроек проекта и правил вашей команды.